DEVELOPER BRIEF — READ THIS BEFORE ANYTHING ELSE

Two systems. Two repositories. Two databases. They never share code and never share a process.

Document	What it is
`00-developer-brief.md`	This file. Principles, shared infrastructure patterns, data, costs.
`01-tennis-build-spec.md`	Tennis system, full spec.
`02-football-build-spec.md`	Football system, full spec.
`03-thirty-day-plan.md`	Day-by-day schedule, deliverables, done-list, blockers log.

You are building for someone who publishes betting selections to an audience. You are not building a bot that places bets. Nothing in these specs requires an account with a bookmaker, and nothing in them should ever place a wager automatically.

---

0. If you read nothing else, read this

You have probably built prediction systems before. This one has a property that most don't, and getting it wrong is the single most common way these projects fail:

> **Predicting the winner correctly is not the objective, and is not even strongly correlated with the objective.**

The objective is to find prices that are *longer than they should be*. A selection that wins 55% of the time at odds of 2.50 is excellent. A selection that wins 78% of the time at odds of 1.25 loses money. Both of those are arithmetic, not opinion:

- 55% at 2.50 → expected value = 0.55 × 2.50 − 1 = **+37.5%**… (illustrative, not a real edge)
- 78% at 1.25 → expected value = 0.78 × 1.25 − 1 = **−2.5%**

So every time you are tempted to optimise for accuracy, hit rate, or "% of matches called correctly" — stop. Those metrics are near-orthogonal to profit and they are trivially gameable by only ever picking short favourites.

Optimise for calibration and for price gap. Those two things, nothing else.

---

1. The honest evidence position

This matters because it determines what the system is *allowed to claim*, and because a developer who doesn't know this will over-build the wrong component.

1.1 What the research actually says about beating these markets

Tennis. Wilkens tested six families of model (logistic regression, neural nets, random forests, gradient boosting, SVMs, ensembles) across roughly 39,000 ATP and WTA matches from 2010–2019. All of them landed around 70% accuracy — which is what you get from the bookmaker's odds alone. The always-back-the-favourite baseline is around 65%. His conclusion was that most of the relevant information is already embedded in the betting market, and that adding further match- and player-specific data produced no significant improvement.

Jeff Sackmann, who runs Tennis Abstract and produces the best free tennis data in existence, publishes his own model losing. His published comparison puts Tennis Abstract at a Brier score of .182, behind FiveThirtyEight at .171 and behind Pinnacle's market prices at .172.

Read that again: the best public tennis model is worse-calibrated than the closing price.

Football. A textbook Dixon-Coles model on public data is a baseline, not an edge. The practitioner who published the canonical replication said plainly that these models will not make you money. Expected goals beats raw goals as a forward predictor (Pearson correlation with future goals 0.71 versus 0.58) — a material gap, but not an enormous one, and the market knows it too.

1.2 What *is* measured

Two things, both structural rather than analytical, and neither requiring any sporting knowledge:

(a) Margin arbitrage against a sharp fair price. Take a sharp bookmaker's price (Pinnacle is the standard reference), strip its margin properly, and you have a good estimate of true probability. Where a softer bookmaker offers a price *longer* than that fair price, you have

positive expected value. Buchdahl measured this: **+3.40% yield across 22,281 bets** at any positive edge, rising to **+13.22% across 1,927 bets** where the edge exceeded 5%. The control — deliberately taking prices *below* fair value — returned **–3.08% over 44,673 bets**, which is exactly what you'd predict, and is what makes the result credible.

(b) Asian handicap line-type selection. Whelan et al., across 84,230 and 67,373 matches, found that the cost of betting differs systematically by handicap line type:

Line type	Loss rate
Integer (0, –1, +1)	**3.24%**
Quarter (.75)	3.57%
Quarter (.25)	3.61%
Half (.5)	**4.16%**

Nearly a full percentage point between the cheapest and most expensive line type, for structurally similar bets. The same work found **no favourite-longshot bias in Asian handicap markets**, unlike 1X2. This is free money in the sense that it costs nothing to prefer integer and .75 lines over .5 lines — and it is not in either sport spec as a "model", it is a **line-selection rule** you apply to every football bet.

1.3 The published-model edges we are building on

These are real, peer-reviewed, and thin. Both specs use them. Neither should be oversold.

- **Wheatcroft (2020), GAP ratings**, *International Journal of Forecasting* 36(3), 916–932 — around **+0.8% profit per bet over twelve years** in the over/under market, using shots and corners as inputs rather than goals. Tested against a random-betting control. The accepted version is free on the LSE research repository.
- **Wheatcroft (2021)**, *Journal of Sports Analytics* — the same rating framework used to predict the match statistics themselves in advance, which is what lets one engine price corners, shots and totals. Free on arXiv (2001.09097).
- **Intransitivity graph neural network (2025)**, arXiv 2510.20454 — applied indiscriminately it returns **–5.49%**. Applied only to high-intransitivity matchups it returns **+3.26% with Kelly staking over 1,903 bets** at 65.7% accuracy and a Brier score of 0.215. The selectivity is the whole result; build the filter, not the model.

1.4 What failed, and why you must not build it

The Knottenbelt common-opponent tennis model published +3.8% ROI over 2,173 ATP

matches. A replication found that figure was driven by **a single outlier bet against erroneously long odds in the test data**. Cleaned up, the strategy is not profitable in practical scenarios. The replicators concluded the residual returns came from exploiting individual bookmakers' mispricing relative to the market — i.e. line shopping — and not from the tennis model.

We still use a common-opponent routing step in the tennis build, but **only as an input to a probability estimate, never as a standalone betting strategy**, and the spec says so.

1.5 The base rate you should hold in your head

Across 1,525 tracked bettors, **60 had positive closing-line value**. About three-quarters of those were not statistically significant. That leaves **roughly 1%** with a defensible edge.

Also: in a *fully efficient* market, across fourteen seasons, there is a **77.63% probability of finding at least one season with statistically significant home bias**. If you go looking for patterns you will find them whether or not they exist. This is the single biggest threat to this project and Section 4 exists to control it.

---

2. Non-negotiable rules

These are not preferences. A build that violates any of them produces numbers that look fine and mean nothing.

R1 — Football model inputs are shots and corners, not goals

This is counterintuitive and you will want to argue with it. Wheatcroft's entire published result depends on it. Goals are the noisy outcome; shots and corners are the repeatable process. Goals may be used for *validation*, never as a GAP rating input.

R2 — De-vig with the power or Shin method, never naive normalisation

Naive multiplicative de-vig (`p = (1/o) / booksum`) leaves longshot probabilities too high, and therefore their fair prices too short — which **manufactures false value exactly where the bookmaker's real margin is hiding**. On a 1.25 / 4.50 / 15.00 book, the implied fair price of the longshot ranges from 16.33 to 27.00 depending on method — a 65% spread — while the favourite's varies by 5.5%.

Rule: methods converge on balanced two-way markets and diverge violently wherever a genuine longshot is involved. **Anything priced above 4.00 must use power or Shin, and the method used must be recorded on the bet record.**

R3 — The margin is not a tax on profit, it is a hurdle on accuracy

At a 5% hold two-way market you're offered about 1.905 each side. Break-even is 52.5%, not 50%. Your probability must beat the market's by 2.5 points before you earn anything. Against a 2% market the hurdle is about 1 point. This is the entire reason venue selection matters more than model quality.

Note also that the headline margin *understates* the real cost. Hegarty & Whelan, on 151,683 football and 130,243 tennis matches: overround-predicted loss rates of 7.1% and 5.4% against **actual** loss rates of **8.7% and 7.5%**. By odds decile the shortest prices cost about 3% and 2.5%; the longest cost about **19% (football) and 25% (tennis)**. Margin is loaded onto longshots.

R4 — Walk-forward validation only

Never fit and test on the same period. Train on years 1–3, test on year 4; roll forward; repeat. Models validated with a simple train/test split showed roughly **12% better backtest performance and then 28% worse forward performance** than cross-validated equivalents. If you produce a single beautiful backtest number over the whole history, you have produced nothing.

R5 — Every data record carries an as-of timestamp

Store the time at which each fact became knowable, and filter on it at prediction time. This is the only reliable defence against look-ahead bias, which is the most common way a backtest lies. If a rating was computed using a match that hadn't finished yet, the whole result is void.

R6 — Log every decision, including the no-bets

Most people only log bets, and then cannot answer "did the model correctly decline?" You need the full slate: every fixture considered, every probability produced, every price seen, the gap, and the fire/no-fire decision with reason.

R7 — Version the model, the features and the calibration together

Every stored prediction carries a `model_version` string. When something is changed in month three, you must be able to say which version produced which results. Without this, three months of data becomes uninterpretable the first time you tune anything.

R8 — Fail loudly, never quietly

If a feed is stale, an API is down, or a rating is older than its permitted window, the system **refuses to publish**. It does not fall back to old numbers. See Section 5.

R9 — Nothing is published without probability, fair price, market price and gap

The pick record schema in Section 6 is mandatory. A selection with no stated probability cannot be evaluated, cannot be calibrated, and is indistinguishable from a guess.

R10 — No language of certainty, anywhere in the codebase or output

Banned in any generated text: banker, lock, guaranteed, certainty, free money, can't lose. A strong selection can lose and a poor selection can win. One result never judges a decision.

---

3. The arithmetic, in one place

Implement these as a single shared library **per repository** (copy it; do not create a shared package that couples the two systems).

Quantity	Formula
Expected value per unit staked	$EV = p \cdot o - 1$
Fair odds from probability	$o_{fair} = 1/p$
Break-even probability from odds	$p_{be} = 1/o$
Booksum	$S = \Sigma(1/o_i)$
Overround	$S - 1$
Hold	$1 - 1/S$
Multiplicative de-vig	$p_i = r_i/S$ where $r_i = 1/o_i$
Additive de-vig	$p_i = r_i - (S-1)/n$
Power de-vig	$p_i = r_i^{(1/k)}$, solve k such that $\Sigma p_i = 1$
Shin de-vig	$p_i = [\sqrt{(z^2 + 4(1-z) \cdot r_i^2/S)} - z] / (2(1-z))$, solve z such that $\Sigma p_i = 1$
Odds-ratio de-vig	$p_i = r_i/(OR + r_i - OR \cdot r_i)$, solve OR
Closing line value as EV	$CLV = (bet_odds / closing_odds) / (1 +$

	`closing_margin) − 1`
Kelly fraction	`f* = (p·o − 1)/(o − 1) = EV/(o − 1)`
Profit variance per bet	`σ² ≈ o − 1`
Standard error of ROI	`SE ≈ √(o−1) / √N`
Bets needed to prove an edge	`N = z²(o−1)/E²`

Note the division by `(1 + closing_margin)` in the CLV formula. Every affiliate explanation of CLV omits it. Beating the closing price by less than the closing margin is not an edge at all, and this is precisely why "I beat the close 60% of the time" is entirely consistent with losing money.

Both power and Shin require a one-dimensional root-find. `scipy.optimize.brentq` over a sensible bracket is fine. Write unit tests that confirm `Σpᵢ = 1` to within 1e-9 for both.

3.1 How long before you know anything

`N ≈ z²(o−1)/E²`:

Odds	Edge	Bets for 95% confidence	Bets for 99%
1.91	3%	2,725	5,449
2.00	3%	**3,004**	6,006
2.00	5%	**1,080**	2,159
2.50	5%	1,648	3,295
5.00	5%	4,489	8,976
10.00	5%	10,172	20,337

Three consequences:

1. Sample requirement scales with `(o−1)` — betting longshots multiplies the evidence burden roughly ninefold at 10.00 versus 2.00.
2. It scales with `1/E²` — halving the edge quadruples the bets needed.
3. **At a 100-bet sample, about 40% of genuinely losing bettors are showing a profit.**

This is the reason the whole system is measured on closing-line value and calibration rather than on profit. Outcome variance is roughly 75× closing-line-value variance. What takes thousands of bets to establish from profit and loss can take 50–100 bets from line movement.

---

4. The three measurement layers

Build all three before you build any model. They are the product; the models are replaceable.

4.1 Calibration — "do my numbers mean anything?"

Bin every published probability into deciles (0–10%, 10–20%, … 90–100%). For each bin, plot predicted probability against observed frequency. A well-calibrated model sits on the diagonal.

This is more fundamental than closing-line value. If the model says 60% and it happens 45% of the time, then every edge calculation downstream is wrong — you are comparing a bad probability to a good price and calling the difference value. It will look fine on the pick sheet and bleed money indefinitely.

Report Brier score (mean squared error of the probability) alongside the reliability diagram. Reference points: Pinnacle's tennis market scores .172; the best free public tennis model scores .182. **If your model's Brier score is worse than the market's, it is not a source of edge and the system must lean entirely on price capture.** That is an acceptable outcome. It is not an acceptable thing to hide.

4.2 Closing line value — "am I beating the market?"

For every selection, capture the price taken (with timestamp) and the closing price at the same bookmaker and, separately, at the sharp reference. Compute CLV using the formula in Section 3, **including the margin division**.

Why this is the primary metric: across 1,525 tracked bettors, first-half CLV explained about half the variance in second-half CLV, while raw profit showed complete mean reversion. CLV persists; profit over small samples does not.

Known limits — implement the dashboard so these are visible, don't pretend them away:

- *False negatives.* A genuinely novel model may move no line and show no CLV while still being right.

- *False positives.* Latency arbitrage and steam-following produce impeccable CLV with zero forecasting skill.
- CLV inherits the closing line's own biases and its de-vig ambiguity. On longshots it is close to arbitrary.
- Measurable is not the same as actionable: a published tip whose price collapses on publication has excellent CLV and no accessible edge.

4.3 The decision log — "what did the system do and why?"

Every fixture on every slate, whether or not it produced a selection. Schema in Section 6.

---

5. Shared infrastructure patterns

Implement separately in each repository. Same design, no shared runtime.

5.1 Storage

PostgreSQL. Not spreadsheets, not CSV files, not SQLite once you're past week one. You will run thousands of backtests and you need indexed queries and transactional integrity.

Core tables (names indicative):

- `fixtures` — the slate. One row per event considered.
- `odds_snapshots` — every price observed, every bookmaker, every market, with `observed_at`. Append-only. Never update a row.
- `ratings` — model state over time, with `as_of` and `model_version`.
- `predictions` — one row per fixture per market per model version.
- `decisions` — fire / no-fire with reason code.
- `settlements` — outcomes, linked back to predictions.
- `clv` — computed after close, joined to decisions.
- `data_health` — feed freshness heartbeats.

5.2 Scheduling

A scheduler (Airflow, Prefect, Dagster, or plain cron plus a job table if you prefer — this does not need to be sophisticated). Jobs:

Job	Cadence
Ingest historical results	Daily
Ingest odds snapshots	Every 15–30 min for fixtures inside the publication window
Recompute ratings	After each results ingest
Generate slate	Per the sport spec's schedule
Generate predictions	Per the sport spec's schedule
Capture closing prices	At event start time
Settle	Hourly
Compute CLV and calibration	Daily
Health check and alerting	Every 15 min

5.3 Data freshness gate — mandatory

Before any slate is published, the system checks the age of every input feed against a configured maximum. If any feed is older than its window:

- The slate is **not published**.
- An alert fires naming the stale feed and its age.
- The failure is written to `data_health`.

There is no fallback to cached data. A model priced off yesterday's numbers will confidently produce selections nobody can diagnose later. Suggested windows: odds ≤ 30 minutes, results ≤ 24 hours, injury and team news ≤ 6 hours.

5.4 Monitoring and alerting

- **Drift alert.** If rolling CLV over the last 30 selections is negative for 14 consecutive days, email. This is the early-warning system for a broken feed or stale ratings.
- **Calibration alert.** If the reliability diagram's largest bin deviation exceeds 10 percentage points over the last 200 predictions, email.
- **Volume alert.** If the slate produces zero selections for 7 consecutive days in a sport, email — that usually indicates a threshold or de-vig bug, not a quiet market.
- **Job failure alert.** Any scheduled job that fails or does not run.

5.5 Kill switch — mandatory, written down before launch

Two independent stops, either of which halts publication until a human reviews:

1. **Closing-line-value stop.** After 200 settled selections in a sport, if cumulative CLV is negative, publication stops. Not "we review it" — it stops.
2. **Drawdown stop.** A pre-agreed notional drawdown on the paper ledger halts publication.

The reason this must be written now rather than later: without a pre-committed number, a bad run is always rationalised. Note the counter-risk honestly — a working model with 6,562 selections (expected 4.19% yield, actual 3.79%, p = 0.838) still threw off 1,000-bet windows that looked like 1-in-40 and 1-in-60 failures. **A drawdown alone never diagnoses a model.** That is why the primary stop is CLV, which is far less noisy.

5.6 Paper trading mode

Every selection is generated, published to an internal ledger, priced, and settled — with no money involved — from day one. This is the default mode and it is never turned off. Live use sits alongside it, not instead of it.

---

6. The pick record schema

Every selection and every declined fixture produces one of these. This is the contract between the system and everything downstream.

```
{
  "record_id": "uuid",
  "model_version": "tennis-1.4.2",
  "sport": "tennis",
  "competition": "ATP 250 Adelaide",
  "fixture_id": "…",
  "event_start_utc": "2026-01-08T00:30:00Z",
  "market": "match_winner",
  "selection": "Player A",

  "model_probability": 0.601,
  "model_probability_interval": [0.548, 0.652],
  "fair_odds": 1.664,

  "reference_book": "pinnacle",
  "reference_price_raw": 1.83,
  "reference_booksum": 1.0241,
```

```
  "devig_method": "power",
  "reference_fair_probability": 0.5642,
  "reference_fair_odds": 1.7724,

  "best_available_price": 1.95,
  "best_available_book": "…",
  "price_observed_at": "2026-01-07T22:04:11Z",
  "minimum_acceptable_price": 1.88,

  "edge_vs_reference": 0.0637,
  "threshold_applied": 0.05,
  "decision": "FIRE",
  "decision_reason": "edge_above_threshold",

  "agent_outputs": { "…": "…" },
  "veto_checks": { "…": "…" },

  "suggested_stake_fraction": 0.0079,
  "staking_rule": "quarter_kelly_cap_2pct",

  "closing_price": null,
  "closing_booksum": null,
  "clv": null,
  "settled_result": null
}
```

On what gets published to the audience. The system's output is a probability, a fair price, a market price and a gap. Whatever the presentation layer does with that, the underlying record must contain all four — including for declined fixtures. A no-bet is information, and a channel that can say "nothing today, here's why" is doing something almost nobody in this space does.

Do not build a component that generates a narrative explanation which is not derived from the model's actual inputs. If the model fired because the price was 6% long, that is the reason. A generated sentence attributing the selection to something the model never considered is not a simplification, it's a different claim.

---

7. Data sources

7.1 Free, and doing most of the work

Source	What it gives	Notes

football-data.co.uk	Historical results, shots, shots on target, corners, cards, referee, plus opening and closing odds from around 20 bookmakers. English pyramid down to League Two, plus the major European leagues. ~25 seasons.	CSV download, no key. **This is the backtest spine for football.** No xG.
Jeff Sackmann / `tennis_atp` on GitHub	ATP match results and match-level stats back decades, including Challenger level. Serve points won, return points won, aces, double faults.	CSV in git, no key. **This is the backtest spine for tennis.**
Sackmann Match Charting Project	Point-level and shot-level data.	Coverage is uneven — skewed to Slams, finals and famous players. Thin exactly where Challenger analysis needs it. Do not assume coverage; check per-player.
tennis-data.co.uk	Historical tennis odds.	Pairs with Sackmann for backtesting.
FBref	Deeper football stats including xG for major leagues.	Scraping is fragile and rate-limited; respect their terms. Not available for League One / League Two.
Understat	xG for top European leagues only.	Explicitly does not cover the English lower divisions.

Important limitation to design around: there is **no public xG for League One and League Two**. football-data.co.uk carries results, shots, corners, cards, referee and odds for those divisions but not xG. This is fine — the GAP engine runs on shots and corners, which are present. It just means the xG sanity check in the football spec applies only to the top-flight leagues.

7.2 Paid, and where the money should go

Verify current pricing yourself before committing — these move, and the figures below are estimates from mid-2026, not quotes.

Source	Purpose	Rough monthly	Priority
Odds feed (The Odds API,	Live prices across many	£30–£150 depending on call	

OddsJam, or equivalent)	bookmakers, including the sharp reference.	volume and book coverage	**Essential**
Betfair Exchange API	Exchange prices; a usable sharp reference and a good closing-line source.	Free API, requires a funded account	**Essential if Pinnacle is unavailable**
Sportmonks or API-Football	Football fixtures, lineups, injuries, live data.	£30–£80	High
Tennis live data (Sportradar, Sportmonks)	Order of play, withdrawals, live scores.	£30–£50	Medium
Opta / Stats Perform	Gold-standard football event data.	Enterprise pricing	Not now

Total realistic monthly for a serious setup: roughly £200–£300. The odds feed is the item worth paying for; everything else can start free.

7.3 The access problem you need to solve early

The sharp reference price is the single most important input to both systems, and **Pinnacle does not accept UK customers**. You do not need an account to bet — you need access to their prices as data. Resolve this in week one, before building anything on top of it. Options, in order of preference:

1. An odds data provider that carries Pinnacle's prices as a feed.
2. Betfair Exchange as the sharp reference instead, with commission accounted for. Effective odds on the exchange are `o_eff = 1 + (o − 1)(1 − c)` where `c` is the commission rate. Use `o_eff` everywhere, never the displayed price.
3. Another low-margin book as reference, with its margin measured rather than assumed.

Whichever you choose, the reference book's identity and its measured booksum go on every pick record. Do not hardcode an assumed margin.

7.4 Free papers — read these, don't just implement from this spec

- Wheatcroft, *A profitable model for predicting the over/under market in football* — accepted version free on the LSE research repository.
- Wheatcroft, *Forecasting football matches by predicting match statistics* — free on arXiv, identifier 2001.09097.

- *Intransitive Player Dominance and Market Inefficiency in Tennis Forecasting: A Graph Neural Network Approach* — free on arXiv, identifier 2510.20454.
- Constantinou, *Investigating the efficiency of the Asian handicap football betting market with ratings and Bayesian networks* — free on arXiv, identifier 2003.09384. Useful because it tested **all** betting decision thresholds rather than picking one, and compared average against best-available odds.
- Dixon & Coles (1997), *Modelling Association Football Scores and Inefficiencies in the Football Betting Market*, JRSS-C 46(2) — the source of the low-score correction.
- Barnett & Clarke, *Combining player statistics to predict outcomes of tennis matches*, IMA Journal of Management Mathematics 16(2):113 — the tennis hierarchical model.

There is also a published Python walkthrough of a GAP ratings implementation on Medium ("Implementing the GAP ratings in python"). Useful as a structural reference; verify it against the paper rather than trusting it.

---

8. Staking

The system publishes a suggested stake fraction. It does not place bets.

```
f* = (p·o − 1)/(o − 1)
```

Use quarter Kelly, capped at 2% of bankroll. Not full Kelly, and not because full Kelly is aggressive — because *the input is an estimate*. Simulations at a true 2.5% expected value:

- Correctly sized full Kelly: about a 34% chance of ending in loss (versus 18% flat-staking).
- Believing the edge is 5% when it is 2.5%: ahead only half the time, with an 8%+ chance of a 90% drawdown.
- **Overestimating threefold: more likely to lose (66%) than to profit (34%) — with a genuine, real edge.**

Since the whole premise of this project is that the edges are thin and the probability estimates are uncertain, systematic overestimation is the expected failure mode. Quarter Kelly with a hard cap is the honest response.

---

9. Things that must not go in

Listed because they are the obvious things to add and each one would actively harm the build.

- **Streak and form rules** — "scored in five straight", "unbeaten in six". These are the most trivially computable statistics in sport, every bookmaker's model contains them, and they are fully in the price before you start. Worse, they are backward-looking outcome noise, which is the exact thing GAP ratings exist to replace with process. Not as an input, not as a tiebreaker, not as a weighting.
- **Head-to-head records in tennis.** Sackmann's finding is blunt: incorporating head-to-heads into pre-match forecasts barely moves the needle. Three compounding problems — tiny samples, a selection bias (players meet *because* both are winning), and staleness.
- **ATP ranking as a predictive variable.** It is a 52-week achievement-and-availability ledger: no opponent adjustment, monotonic in participation, a 12-month cliff, surface-blind, discrete and lagged. Elo fixes precisely these. Use Elo.
- **"Due a win" logic of any kind.** Gambler's fallacy.
- **Multi-threshold rule search.** Testing five, six and seven-game windows to see which backtests best manufactures false rules. Searching around ten variants produces an expected Sharpe ratio near 1.57 from pure noise. If you must test variants, **count them and record the count** so the significance can be adjusted.
- **Anything derived from goals as a GAP input.** See R1.

---

10. What "done" looks like for each component

Component	Done when
De-vig library	Power and Shin both root-find to `Σp = 1` within 1e-9; unit tests cover 2-way, 3-way and a longshot book; method recorded on output
Data ingest	Full history loaded, `as_of` populated on every row, re-running is idempotent
Ratings engine	Reproduces a known published figure or a hand-computed example; deterministic given the same input and cutoff
Backtest harness	Walk-forward only; refuses to run if any feature's `as_of` post-dates the fixture; reports variant count
Slate	Produces every fixture in scope, every day, with no filtering by attractiveness

generator	
Manager	Fires only on price gap; declines are logged with reason
CLV pipeline	Captures closing price automatically; formula includes the margin division
Calibration	Reliability diagram plus Brier score, by market and by model version
Health gate	Blocks publication on stale feeds; alert fires; tested by deliberately staling a feed
Kill switch	Both stops implemented and tested by injecting a synthetic losing run